

ECOM EXPERTS

Ecom discountlab

Manual de usuario, tecnico y de desarrollo

ecom_discountlab 1.0.9 · 07/09/2026

Contenido

1. Qué te da este módulo	
2. Instalación	
3. Antes de nada: el precio de coste	
4. Crear tu primera promoción	
5. Recetas	
5.1 Un 3×2	
5.2 Descuento por cantidad	
5.3 Un pack	
5.4 Segunda unidad al 50 %	
5.5 «20 % pero como mucho 30 €»	
5.6 Solo clientes VIP, de noche, de lunes a jueves	
5.7 Solo primera compra	
6. Leer el ROI	
7. El simulador	
8. Canibalización	
9. Lotes de cupones	
10. En el front	
11. Problemas frecuentes	
12. Si algo falla	
1. Qué hace	
2. Hooks utilizados (verificados en 9.1.1)	
2.1 Grid de descuentos	
2.2 Formulario de descuento — los DOS juegos, a propósito	
2.3 Motor de carrito	
2.4 Métricas y front	
3. Estructura	
4. Base de datos	
4.1 Cómo se alimentan las métricas	
5. El motor	
5.1 Tipos	
5.2 Normalizadores	

6. Configuración

7. Traducciones

8. Autocomprobación de arranque

9. Decisiones de diseño y por qué

10. Verificación realizada

1. Mapa mental en treinta segundos

2. Añadir un tipo de regla nuevo

3. Cómo depurar

3.1 Activar los logs

3.2 «No pasa nada»: usa el panel de salud, no el código

3.3 Reproducir el cálculo sin tienda

4. Cosas que NO hay que hacer aquí

5. Regenerar las traducciones

6. Migraciones

7. Comprobaciones antes de dar por buena una versión

Versión 1.0.9 · Requiere PrestaShop 9.0 o superior

1. Qué te da este módulo

PrestaShop sabe hacer «10 % de descuento» y poco más. Este módulo añade:

- **3×2, 2×1 y cualquier N×M**
- **Descuento por cantidad** (5 uds → 5 %, 10 uds → 10 %, 20 uds → 15 %)
- **Packs:** compra A + B + C y el conjunto te sale por un precio fijo
- **Segunda unidad al 50 %** (o al porcentaje que quieras)
- **Topes:** «20 % de descuento, pero como máximo 30 €»
- **Condiciones cruzadas:** solo clientes VIP, solo de 22:00 a 02:00, solo de lunes a jueves, solo primera compra...
- **Lotes de cupones únicos** de un solo uso, exportables a CSV

Y sobre todo, lo que no da nadie: **te dice si cada promoción gana o pierde dinero.**

2. Instalación

1. Sube el ZIP desde **Módulos** → **Subir un módulo**.
2. Pulsa **Configurar**.
3. Baja al panel **Autocomprobación de arranque**, al final de la pantalla. Si sale todo en verde, ya está funcionando. Si hay avisos, dicen exactamente qué falta.

El módulo se cuelga del menú en **Catálogo** → **Descuentos** → **Discount Lab**, con tres subpáginas: Reglas avanzadas, Simulador y Lotes de cupones. La tabla *Menú del back-office* del panel de autocomprobación te dice si las cuatro pestañas están colgando donde deben; si alguna aparece «sin padre», vuelve a entrar en la configuración y se re-engancha sola.

Al instalar, el módulo calcula de una vez las estadísticas de las promociones que ya tenías usadas en pedidos anteriores. A partir de ahí se actualizan solas, pedido a pedido.

3. Antes de nada: el precio de coste

El módulo calcula margen y ROI restando el **coste de mercancía**. Si tus productos no tienen precio de coste, verás «sin dato» en gris. **Es a propósito: nunca se inventa un ROI.**

Dos salidas:

- Rellena el precio de coste de los productos (ficha de producto → Precios → Precio de coste), o
 - Configuración → pestaña **ROI** → Base de coste = *Porcentaje fijo sobre la venta*, e indica tu margen medio (por ejemplo 60 si tu coste es el 60 % del precio de venta).
-

4. Crear tu primera promoción

Todo se hace desde un solo sitio: **Catálogo** → **Discount Lab** → **Reglas avanzadas** → **Nueva regla**.

1. **Promoción asociada:** deja marcado «+ **Crear una promoción nueva**» (viene marcado por defecto) y rellena el bloque que aparece debajo: - **Nombre:** lo que verá el cliente en el carrito. - **Código:** déjalo **en blanco** si quieres que la promoción se aplique sola en cuanto el carrito cumpla la regla (un 3×2 normalmente va así). Ponle código —o pulsa **Generar**— si el cliente tiene que teclearlo. - **Vigencia**, usos totales, usos por cliente y prioridad.
2. **Tipo de regla:** el que quieras.
3. **Ámbito:** todo el carrito, o solo unos productos / categorías / marcas (IDs separados por comas).
4. Rellena el bloque del tipo elegido y **Guardar**.

Al guardar se crean de golpe las dos cosas: la promoción y su regla. Ya está activa.

Por qué no se crea en Catálogo → Descuentos. El formulario de PrestaShop **obliga a poner una acción** (un porcentaje, un importe, portes gratis o un regalo) y da el error «*Una acción es requerida para esta regla de carrito*». Las promociones de este módulo no llevan ninguna de esas acciones a propósito: **el importe lo calcula el motor del módulo** (un 3×2 no es un porcentaje fijo, depende de lo que haya en el carrito). Por eso la promoción se crea desde aquí, donde esa validación no aplica.

Si ya tenías una promoción creada a mano y quieres reaprovecharla, elígela en el desplegable bajo «**Usar una existente**». Solo salen las vigentes que aún no tienen regla avanzada.

5. Recetas

5.1 Un 3×2

- Tipo: **N x M**
- Se lleva **3**, Paga **2**
- *Agrupar por producto* **activado** si quieren ser tres unidades del mismo artículo; **desactivado** si vale cualquier combinación dentro del ámbito.
- Ámbito: normalmente *Categorías* con la categoría en promoción.

La unidad gratis es siempre **la más barata** del grupo, que es lo que espera el cliente.

5.2 Descuento por cantidad

- Tipo: **Escalado por cantidad**
- Añade un tramo por línea: Desde **5** → **5** → **%**; Desde **10** → **10** → **%**; Desde **20** → **15** → **%**.
- *Contar por producto* activado: cuenta unidades del mismo artículo. Desactivado: suma todas las unidades del ámbito.

Los tramos se ven solos en la ficha de producto si tienes activada esa opción.

5.3 Un pack

- Tipo: **Pack de productos**
- IDs de producto del pack: 15,22,31
- Precio del pack: 49,90 (marca *con impuestos* si ese precio es el final)

Si el carrito lleva los tres productos, el conjunto pasa a costar 49,90 €. Si lleva dos de cada uno, se forman dos packs.

5.4 Segunda unidad al 50 %

- Tipo: **Segundo artículo al X %**
- Descuento sobre la 2ª unidad: 50

5.5 «20 % pero como mucho 30 €»

- Tipo: **Tope máximo de descuento**
- Porcentaje: 20 , Tope: 30

5.6 Solo clientes VIP, de noche, de lunes a jueves

- Tipo: **Condiciones cruzadas**
- Grupos de cliente: VIP · Franja horaria: 22:00 – 02:00 · Días: L M X J

Este tipo **no calcula importe**: deja que el descuento del núcleo aplique su porcentaje, pero solo cuando se cumplen las condiciones.

5.7 Solo primera compra

- Tipo: **Condiciones cruzadas**
- Pedidos previos del cliente: mínimo vacío, máximo 0

6. Leer el ROI

Catálogo → **Discount Lab** te da el panel de rendimiento y el ranking. Las mismas columnas aparecen en la lista de Descuentos de PrestaShop.

Columna	Qué significa
Usos	Pedidos válidos con esa promoción
Ingresos	Facturación de esos pedidos, sin impuestos
Ticket medio	Ingresos ÷ usos
Descuento	Lo que has regalado
Margen	Ingresos – coste – descuento
ROI	Margen ÷ descuento

El semáforo:

-  **Verde** ($\text{ROI} \geq 2$): por cada euro de descuento ganas dos o más de margen. Repítela.
-  **Ámbar** (entre 1 y 2): se sostiene, pero justa.
-  **Rojo** (menos de 1): **la promoción te está costando más de lo que deja.**
-  **Gris**: sin coste de mercancía, no se puede calcular (ver apartado 3).

Los umbrales se cambian en Configuración → pestaña ROI.

Los pedidos cancelados, reembolsados o con error de pago se descuentan solos.

7. El simulador

Catálogo → Discount Lab → Simulador.

Elige una regla y un periodo pasado, y te dice qué habría pasado si esa promoción hubiera estado activa: cuántos pedidos la habrían activado, cuánto descuento habrías dado y qué ROI habrías sacado.

No toca nada. No modifica pedidos, no guarda nada y no envía nada a ningún sitio. Es la forma de probar un 3×2 antes de lanzarlo.

8. Canibalización

Cuando dos promociones se solapan a la vez **en fechas y en productos**, un mismo artículo puede llevarse los dos descuentos. Suele descubrirse cuando ya está cobrado.

El módulo avisa al guardar una regla y en el dashboard. Desde Configuración → pestaña **Canibalización** puedes hacer que además **bloquee** el guardado en vez de solo avisar.

Si tienes muchas promociones sin restricción de producto, activa también *Avisar de promociones globales*; si no, el módulo asume que ese caso ya lo conoces.

9. Lotes de cupones

Catálogo → Discount Lab → Lotes de cupones.

1. Elige una **promoción plantilla** (un descuento normal, con sus condiciones ya puestas).
2. Cantidad, prefijo, longitud y fecha de caducidad.
3. **Generar lote.**

Se crean tantos códigos únicos como hayas pedido, cada uno de un solo uso, con las mismas condiciones que la plantilla. Descarga el **CSV** para mandarlos a tu plataforma de email o imprimirlos.

La tabla te dice cuántos se han canjeado. Al borrar un lote se borran también sus descuentos.

10. En el front

Si están activadas esas opciones:

- **Ficha de producto:** tabla con los tramos de descuento por cantidad.
 - **Carrito:** en cada línea, «Añade 1 más y pagas solo 2 de cada 3».
 - **Pie del carrito:** resumen de las promociones que se están aplicando.
-

11. Problemas frecuentes

«**No veo las columnas de ROI en la lista de Descuentos.**» Esas columnas solo existen en PrestaShop 9. En 8.x el motor de reglas funciona igual, pero la lista de Descuentos es la antigua. Consulta el ROI en Catálogo → Discount Lab.

«**El descuento no se aplica.**» Comprueba, por este orden: que la promoción del núcleo esté activa y en fechas; que la regla avanzada esté activa; que los IDs del ámbito sean correctos; y que el carrito llegue al mínimo (un 3×2 con 2 unidades no descuenta nada, es correcto). En el panel de autocomprobación, el punto *Carrito*: *actionGetCartRuleContextualValue* debe estar en verde.

«**He cambiado algo y no se refleja.**» Es la caché. Parámetros avanzados → Rendimiento → **Vaciar caché**. No es que el fichero no se haya subido.

«**El ROI sale en gris.**» No hay precio de coste. Apartado 3 de esta guía.

«**Sale un aviso de multitienda.**» La configuración del módulo es única para todas las tiendas (el back-office es uno solo). Las estadísticas sí se guardan separadas por tienda.

12. Si algo falla

Configuración → pestaña **Depuración** → *Activar logs*, nivel **Debug**. Reproduce el problema y manda el fichero de `modules/ecom_discountlab/logs/ecom_discountlab-AAAA-MM-DD.log`.

Soporte: **ecomyseo@gmail.com**

ecom_discountlab — Documentación técnica

Versión: 1.0.9 · **PrestaShop:** 9.0 – 9.1 (el motor necesita hooks que no existen en 8.x) · **PHP:** 7.4 – 8.4 **Autor:** Ecom Experts · ecomyseo@gmail.com · AFL-3.0 · 2026

1. Qué hace

Añade sobre el motor `Discount` de PrestaShop 9.1 dos cosas que el núcleo no cubre:

1. **Reglas de promoción que Cart Rules no sabe hacer:** N×M (3×2), escalados por cantidad, packs a precio fijo, segunda unidad al X %, topes de descuento y condiciones cruzadas (grupo × marca × franja horaria × día × pedidos previos).
2. **ROI por promoción:** ingresos, coste de mercancía, margen y retorno, con semáforo, directamente en el grid nativo de Descuentos.

2. Hooks utilizados (verificados en 9.1.1)

2.1 Grid de descuentos

Hook	Punto de despacho	Parámetros
<code>actionDiscountGridDefinitionModifier</code>	<code>AbstractGridDefinitionFactory.php:58</code>	<code>definition</code>
<code>actionDiscountGridQueryBuilderModifier</code>	<code>DoctrineGridDataFactory.php:47</code>	<code>search_query_builder</code> , <code>count_query_builder</code> , <code>search_criteria</code>
<code>actionDiscountGridDataModifier</code>	<code>GridFactory.php:43</code>	<code>data</code> (por referencia)
<code>actionDiscountGridFilterFormModifier</code>	<code>GridFilterFormFactory.php:69</code>	<code>filter_form_builder</code>
<code>actionDiscountGridPresenterModifier</code>	<code>GridPresenter.php:77</code>	<code>presented_grid</code> (por referencia)

El nombre se genera con `'action' . Container::camelize($definition->getId()) . '...'` y `DiscountGridDefinitionFactory::GRID_ID = 'discount'`.

El grid consulta `ps_cart_rule` con alias `cr` y expone el id como `id_discount` (`DiscountQueryBuilder.php:38`). Sobre ese alias se hace el `leftJoin` a las tablas del módulo.

2.2 Formulario de descuento — los DOS juegos, a propósito

PrestaShop genera el nombre desde el *block prefix* del FormType:

```
Builder/FormBuilder.php:149 'action' . camelize($formBuilder->getName()) . 'FormBuilderModifier'
Handler/FormHandler.php:150 'actionAfterCreate' . camelize($form->getName()) . 'FormHandler'
```

El formulario nuevo es `PrestaShopBundle\Form\Admin\Sell\Discount\DiscountType`, cuyo block prefix es `discount` → los hooks reales son `actionDiscountFormBuilderModifier` y `actionAfterCreateDiscountFormHandler`. Pero `hook.xml` de 9.1 declara los de `CartRule`, heredados de la página vieja.

Por eso el módulo registra los dos juegos y el panel de salud dice cuál se dispara de verdad en cada instalación (`EcomDiscountHealth::report()['form_hook_verdict']`). Se decide con el dato, no con la suposición.

2.3 Motor de carrito

Hook	Punto	Contrato
<code>actionValidateCartRule</code>	<code>CartRule.php:747</code>	<code>isValidatedByModules</code> por referencia: <code>null</code> sigue la validación del núcleo, <code>true / false</code> la cortan
<code>actionGetCartRuleContextualValue</code>	<code>CartRule.php:1415</code>	<code>contextualValueFromModules</code> por referencia; si no es <code>null</code> , ese es el importe
<code>actionApplyCartRule</code>	<code>CartRuleCalculator.php:116</code>	<code>cart_rule_calculator</code> , <code>cart_rule_data</code> , <code>is_applied_by_modules</code> por referencia

Los dos últimos van SIEMPRE en pareja y con el mismo valor. El propio núcleo lo advierte en `CartRule.php:1410`. En el módulo los dos llaman a `EcomDiscountEngine::computeCached()`; si se duplicase la lógica, el total del carrito y el del pedido dejarían de cuadrar.

2.4 Métricas y front

```
actionValidateOrder , actionOrderStatusPostUpdate , actionObjectCartRuleDeleteAfter ,
actionAdminDuplicateDiscountAfter , displayProductPriceBlock , displayCartExtraProductInfo (PS 8.2+),
displayShoppingCartFooter , actionFrontControllerSetMedia , actionAdminControllerSetMedia ,
displayBackOfficeHeader .
```

3. Estructura

```

ecom_discountlab/
  ecom_discountlab.php      clase Ecom_Discountlab, global namespace, sin namespace
  logo.png
  classes/
    EcomDiscountLogger.php  logs en /logs, solo si ECOM_DL_DEBUG
    EcomDiscountRule.php    ObjectModel de la regla avanzada
    EcomDiscountEngine.php  cálculo: nxm, tiers, bundle, second_at, cap, cross
    EcomDiscountMetrics.php consolidación incremental + seed + ROI
    EcomDiscountCannibal.php detector de solapes
    EcomDiscountSimulator.php simulación retroactiva (solo lectura)
    EcomDiscountCoupons.php lotes de cupones únicos
    EcomDiscountGridInjector.php los 5 hooks de grid
    EcomDiscountFormInjector.php los hooks de formulario + duplicado
    EcomDiscountHealth.php  autocomprobación de arranque
  controllers/admin/
    AdminEcomDiscountLabController.php  dashboard, ROI, calendario
    AdminEcomDiscountRulesController.php CRUD de reglas
    AdminEcomDiscountSimulatorController.php simulador
    AdminEcomDiscountCouponsController.php lotes de cupones
  controllers/front/      (reservado, solo index.php)
  views/css/ views/js/ views/templates/{admin,hook}/
  translations/es-ES.xlf translations/en-US.xlf
  sql/install.sql
  logs/ (con .htaccess e index.php)
  docs/
  index.php en CADA directorio, recursivo

```

Sin `src/`, sin `config/`, sin `composer.json`, sin `vendor/`, sin `upgrade/`. El grid y el formulario se extienden por hook: no hace falta un solo servicio Symfony. Migraciones en `addnewfeatures()` con `try {}`.

4. Base de datos

Tabla	Clave	Para qué
<code>ecom_discountlab_rule</code>	<code>id_rule</code>	Regla avanzada, <code>config_json</code> con la configuración del tipo
<code>ecom_discountlab_metric</code>	<code>id_cart_rule</code> + <code>id_shop</code>	Acumulado: usos, ingresos, descuento, coste, cancelados
<code>ecom_discountlab_order</code>	<code>id_order</code> + <code>id_cart_rule</code>	Tabla de control: hace idempotente la consolidación
<code>ecom_discountlab_coupon</code>	<code>id_coupon</code> , <code>code</code> único	Cupones de lote y su seguimiento de uso

4.1 Cómo se alimentan las métricas

Nunca por scan del histórico. Dos caminos y solo dos:

1. **Seed inicial** (`EcomDiscountMetrics::seed()`): se ejecuta **una sola vez**, en su propia transacción, durante `install()` . Agrega de golpe los pedidos válidos preexistentes y guarda la fecha de corte en `seeded_until` y en `ECOM_DL_SEED_DATE` . Solo lee y agrega: **no dispara ninguna acción externa**.
2. **Incremental por evento**: `actionValidateOrder` suma, `actionOrderStatusPostUpdate` resta si el pedido pasa a un estado de la lista de revocación y vuelve a sumar si sale de ella.

La idempotencia la garantiza `ecom_discountlab_order` con PK compuesta (`id_order` , `id_cart_rule`) **más un guard estático en memoria por petición**. No se usa `INSERT IGNORE` + `Affected_Rows()` porque no es fiable con dos hooks concurrentes en la misma petición.

La purga del detalle (`purgeDetail`) borra en bucle con `LIMIT 500` , nunca de golpe.

5. El motor

`EcomDiscountEngine::compute($rule, $rows, $context)` devuelve siempre:

```
array(
  'total_te' => float,    // sin impuestos
  'total_ti' => float,    // con impuestos
  'lines'    => array(    // clave "idProducto-idCombinacion"
    '15-0' => array('id_product' => 15, 'id_product_attribute' => 0,
                  'amount_te' => 9.50, 'amount_ti' => 11.50),
  ),
  'detail'   => array('2 unidades gratis de 6 elegibles'),
)
```

`computeCached()` cachea por `id_rule` + hash del contenido del carrito, así que un mismo carrito no se recalcula dentro de la petición pero un cambio sí lo invalida.

5.1 Tipos

Tipo	Configuración	Cálculo
<code>nxm</code>	<code>buy</code> , <code>pay</code> , <code>per_product</code> , <code>max_groups</code>	Ordena las unidades por precio y regala las más baratas
<code>tiers</code>	<code>tiers[]</code> { <code>from</code> , <code>value</code> , <code>mode</code> } , <code>per_product</code>	Elige el tramo aplicable y descuenta % o importe por unidad
<code>bundle</code>	<code>bundle_products[]</code> , <code>pack_price</code> , <code>pack_price_tax_incl</code> , <code>max_packs</code>	Cuenta packs completos y reparte el descuento proporcionalmente
<code>second_at</code>	<code>percent</code> , <code>per_product</code> , <code>max_pairs</code>	Por cada pareja, descuenta sobre la unidad más barata
<code>cap</code>	<code>percent</code> , <code>cap</code>	Aplica el % y lo escala si supera el tope
<code>cross</code>	<code>groups</code> , <code>manufacturers</code> , <code>hour_from/to</code> , <code>weekdays</code> , <code>min/max_previous_orders</code>	No calcula importe: solo valida

Todos aceptan además `scope` (`all` / `products` / `categories` / `manufacturers`), `scope_ids[]` y `global_cap` .

5.2 Normalizadores

- `normalizeCartProducts()` — desde `Cart::getProducts()` (usado en los hooks de validación y valor contextual).
- `normalizeCartRows()` — desde los `CartRow` del calculador, usando `getInitialUnitPrice()` (usado en `actionApplyCartRule` , donde las filas pueden venir ya descontadas por otra promoción).

6. Configuración

Un solo HelperForm con pestañas y un único botón Guardar: General · ROI · Reglas · Cupones · Canibalización · Depuración. Los paneles que no son formulario (salud, avisos) van en `$this->display()` aparte, con su propio `<form>` .

Todas las claves usan `Configuration::getGlobalValue` / `updateGlobalValue` : **la configuración es GLOBAL**. Se avisa en pantalla si `Shop::isFeatureActive()` . Las **métricas sí son por tienda** (`id_shop` en `ecom_discountlab_metric` y en `ecom_discountlab_order`).

Cada campo declarado en `renderForm()` tiene su valor en `getConfigFormValues()` ; si faltase, Smarty lanzaría `Undefined array key` en modo dev.

7. Traducciones

`isUsingNewTranslationSystem()` devuelve `true` . Dominios `Modules.Ecomdiscountlab.Admin` (243 cadenas) y `Modules.Ecomdiscountlab.Shop` (8 cadenas). **251 en total, las 251 con destino en es-ES.xlf y en en-US.xlf** .

En `.tpl` siempre `{l s='...' d='Modules.Ecomdiscountlab.X'}` . En PHP siempre `$this->trans('...', array(), 'Modules.Ecomdiscountlab.X')` . **Cero usos de {l s='...' mod='...'} y cero de Module::l()** : son el sistema legacy y no leen los `.xlf` .

Los `.xlf` se generan con un script determinista (extracción por regex sobre `.tpl` y `.php` + diff contra el diccionario inglés), no a mano. La última ejecución reportó **0 cadenas sin traducción inglesa**.

8. Autocomprobación de arranque

`EcomDiscountHealth` mantiene un contador y una marca de tiempo por punto de enganche (`Configuration` global, prefijo `ECOM_DL_TICK_`). El panel de configuración responde a:

1. ¿Están los hooks registrados en `ps_hook_module` ?
2. ¿Qué juego de hooks de formulario se dispara realmente?
3. ¿Se ha ejecutado `actionGetCartRuleContextualValue` en la última hora?
4. ¿Está hecho el seed? ¿Con qué fecha de corte?
5. ¿Cuántas promociones tienen usos pero no coste de mercancía?

6. ¿Existen las cuatro tablas?

Sin este panel, una regla perfectamente programada a la que no llama nadie pasaría desapercibida: no está en ninguna línea escrita, está en una línea que no se escribió.

9. Decisiones de diseño y por qué

Decisión	Motivo
CRUD de reglas en controlador propio, no en el formulario Symfony del descuento	Los campos de módulo dentro de un formulario del núcleo dependen del form theme y del mapeo. Sin <code>mapped</code> , Symfony no los incluye en <code>getData()</code> , el hook recibe <code>null</code> , la pantalla dice «Actualizado correctamente» y no guarda nada
Registrar los dos juegos de hooks de formulario	<code>hook.xml</code> declara los de CartRule pero el formulario nuevo dispara los de Discount. Se mide en vez de suponer
<code>overrideImageLink</code> /precios formateados en PHP	<code>{displayPrice}</code> no existe en PS 1.7/8/9: da fatal error de Smarty
Cupones como <code>CartRule</code> reales	El front los valida de forma nativa con <code>getIdByCode</code> ; no hay que interceptar el checkout
<code>Db::getValue</code> en vez de <code>Tab::getIdFromClassName</code>	En PS 9 esa función lanza <code>NonUniqueResult</code> con pestañas duplicadas
<code>class_exists()</code> alrededor de todo lo Symfony	Si PrestaShop cambia una clase, el módulo deja de inyectar pero no tumba el back-office
ROI <code>null</code> cuando no hay coste	Un ROI inventado es peor que ningún ROI. Se muestra «sin dato» en gris

10. Verificación realizada

- `php -l` sobre los **30 ficheros PHP** con **PHP 7.4, 8.1, 8.3 y 8.4**: sin errores.
- 0 usos de `nofilter`, 0 de `displayPrice`, 0 de `mod='...'` legacy, 0 `LIMIT 1` en SQL.
- `index.php` presente en los **14 directorios**, recursivamente.
- Guard `if (!defined('_PS_VERSION_')) exit;` en los 30 PHP.
- 0 variables Smarty sin `|escape` ni filtro.
- 251/251 cadenas con destino en los dos `.xlf`.

ecom_discountlab — Guía de desarrollo

Versión: 1.0.9 · AFL-3.0 · Ecom Experts

Para quien vaya a tocar el código: cómo está montado, cómo extenderlo y cómo depurarlo sin perder tiempo.

1. Mapa mental en treinta segundos

BACK-OFFICE	CARRITO	PEDIDO
-----	-----	-----
Grid de Descuentos	actionValidateCartRule	actionValidateOrder
5 hooks -> GridInjector	-> evaluateCross()	-> Metrics::registerOrder()
	-> hasMinimumRequirements()	
Reglas avanzadas		actionOrderStatusPostUpdate
controlador propio -> Rule	actionGetCartRuleContextual	-> revokeOrder / restoreOrder
	-> Engine::computeCached()	
Simulador	actionApplyCartRule	
-> Simulator -> Engine	-> Engine::computeCached()	
	-> cartRow->applyFlatDiscount()	

La regla de oro: `actionGetCartRuleContextualValue` y `actionApplyCartRule` llaman a la *misma* función. Si alguna vez alguien las separa, el total del carrito y el del pedido dejarán de cuadrar y el bug será difícilísimo de encontrar. El propio núcleo lo advierte en `classes/CartRule.php:1410`.

2. Añadir un tipo de regla nuevo

Cinco pasos, todos en sitios evidentes:

1. **Constante** en `EcomDiscountRule`: `php const TYPE_MI_REGLA = 'mi_regla';`
2. **Etiqueta** en `EcomDiscountRule::getTypes()`. Si el tipo calcula un importe, añádelo también a `getValueTypes()` (si solo valida, no).
3. **Cálculo** en `EcomDiscountEngine`: un `computeMiRegla(EcomDiscountRule $rule, array $rows)` privado y su `case` en el `switch` de `compute()`. Devuelve siempre la estructura `total_te / total_ti / lines / detail`; usa `self::addLine()` para acumular.
4. **Mínimo** en `EcomDiscountEngine::hasMinimumRequirements()`, para que `actionValidateCartRule` invalide la promoción cuando el carrito todavía no llega.
5. **Formulario**: un `<div class="panel ecom-dl-type-panel" data-type="mi_regla">` en `views/templates/admin/rule_form.tpl` (el JS de `rules.js` lo muestra y lo oculta solo), su `case` en `AdminEcomDiscountRulesController::buildConfig()` y una línea en `summarize()`.

Después: `php -1`, regenerar los `.xlf` (apartado 5) y comprobar en el panel de salud que `contextual_value` y `apply` siguen marcando en verde.

3. Cómo depurar

3.1 Activar los logs

Configuración → pestaña **Depuración** → *Activar logs*, nivel **Debug**. Los ficheros salen en `modules/ecom_discountlab/logs/ecom_discountlab-AAAA-MM-DD.log`, rotan a los 2 MB y la carpeta tiene `.htaccess` que impide servirlos.

Con nivel Debug, `EcomDiscountEngine::compute()` registra el tipo, el id de la regla y los dos totales en cada cálculo. Es lo primero que hay que mirar ante un «el descuento sale mal».

3.2 «No pasa nada»: usa el panel de salud, no el código

El panel de configuración lista los trece puntos de enganche con su contador y su última ejecución. Diagnóstico en un vistazo:

Síntoma	Qué mirar
No aparecen las columnas de ROI	<code>grid_definition</code> y <code>grid_query</code> en gris → los hooks no están registrados o la tienda no es PS 9
Las columnas salen pero vacías	<code>grid_data</code> en verde y <code>cogs = 0</code> → falta precio de coste; el ROI se muestra «sin dato» a propósito
La regla no descuenta	<code>contextual_value</code> en gris → la regla no está activa, o el descuento del núcleo está caducado, o <code>filterEligible()</code> deja el carrito vacío
Descuenta pero el pedido no cuadra	<code>apply</code> en gris mientras <code>contextual_value</code> está en verde → alguien rompió la pareja de hooks
Guardo un descuento y no pasa nada	Mira <code>form_hook_verdict</code> : dirá si esta instalación dispara <code>Discount</code> o <code>CartRule</code>

3.3 Reproducir el cálculo sin tienda

`EcomDiscountEngine::compute()` es estático y no depende del contexto: se le puede pasar un array de filas a mano desde un script de prueba. Formato de una fila:

```
array(
    'id_product' => 15, 'id_product_attribute' => 0,
    'id_manufacturer' => 3, 'id_category_default' => 11,
    'quantity' => 6, 'unit_te' => 10.00, 'unit_ti' => 12.10,
    'wholesale_price' => 4.20,
)
```

4. Cosas que NO hay que hacer aquí

- **No crear** `src/` ni `config/services.yml`. El grid y el formulario se extienden por hook; no hace falta un solo servicio Symfony y la estructura del proyecto lo prohíbe.

- **No meter campos del módulo en el formulario Symfony del descuento.** Sin `mapped`, Symfony no los incluye en `getData()`, el hook de guardado recibe `null`, la pantalla dice «Actualizado correctamente» y no guarda nada. Por eso hay un controlador propio.
- **No usar** `{1 s='...' mod='...'} ni $this->module->l()`. Son el sistema legacy y no leen los `.xlf`. Solo `d='Modules.Ecomdiscountlab.X'` y `$this->trans(...)`.
- **No usar** `{displayPrice}`. No existe en PS 1.7/8/9: fatal error de Smarty. Formatea en PHP con `EcomDiscountGridInjector::money()` y pasa la cadena hecha.
- **No añadir** `LIMIT 1` a las consultas: `getRow()` y `getValue()` ya lo ponen.
- **No usar** `Tab::getIdFromClassName()`: en PS 9 lanza `NonUniqueResult` con pestañas duplicadas. Usa `Db::getValue`.
- **No recorrer el histórico de pedidos en caliente.** Las métricas se alimentan por evento; el acumulado antiguo es cosa del seed, que corre una sola vez.
- **No quitar los** `class_exists()` de `EcomDiscountGridInjector`. Son lo que evita que un cambio de PrestaShop tumbé el back-office entero.

5. Regenerar las traducciones

Nunca a mano. El script determinista hace la extracción y el diff:

1. Extrae `{1 s='...' d='Modules.Ecomdiscountlab.X'}` de los `.tpl` y `$this->trans('...', ..., 'Modules.Ecomdiscountlab.X')` de los `.php`.
2. Cruza contra el diccionario inglés.
3. Escribe `translations/es-ES.xlf` (destino = origen) y `translations/en-US.xlf`.
4. **Imprime las cadenas sin traducción inglesa.** Si esa lista no sale vacía, la entrega no está terminada.

El `.xlf` casa por `<source>` y `resname` **exactos**: respetan mayúsculas, comillas y `&`. Cambiar una coma en el código obliga a regenerar.

Estilo del proyecto: **español sin tildes (ASCII)** en los textos de interfaz.

6. Migraciones

No hay carpeta `upgrade/`. Todo va en `Ecom_Discountlab::addnewfeatures()`, que se ejecuta al entrar en la configuración y hace, dentro de un `try {}`:

1. Registra los hooks que falten (los añadidos en versiones posteriores).
2. Crea las tablas que falten.
3. Añade columnas nuevas, **comprobando** `INFORMATION_SCHEMA` **antes de cada** `ALTER`. Es obligatorio: si el `ObjectModel` carga una columna que aún no existe, el `SELECT` peta, y peta también en el front.
4. Crea las pestañas que falten.
5. Escribe la configuración nueva con sus valores por defecto.
6. Ejecuta el seed si está pendiente.
7. Purga el detalle antiguo.

8. Sube `ECOM_DL_SCHEMA` a `SCHEMA_VERSION`.

Para publicar una versión nueva con cambios de esquema: sube `SCHEMA_VERSION`, añade la columna al array `$columns` de `addnewfeatures()` y al `sql/install.sql`. Nada más.

7. Comprobaciones antes de dar por buena una versión

```
php -l    sobre todos los .php, con 7.4 y con 8.x
grep      nofilter / displayPrice / mod=' / LIMIT 1    -> deben dar 0
find      index.php en cada directorio                -> sin huecos
grep      _PS_VERSION_ en cada .php                  -> sin huecos
script    de traducciones                             -> 0 cadenas sin traducir
panel     de salud tras instalar                      -> sin avisos rojos
prueba    real: carrito con 6 unidades y regla 3x2    -> el total del pedido = el del carrito
```

Ese último punto es el que más veces se salta y el único que detecta que la pareja `contextual_value` / `apply` se ha desincronizado.